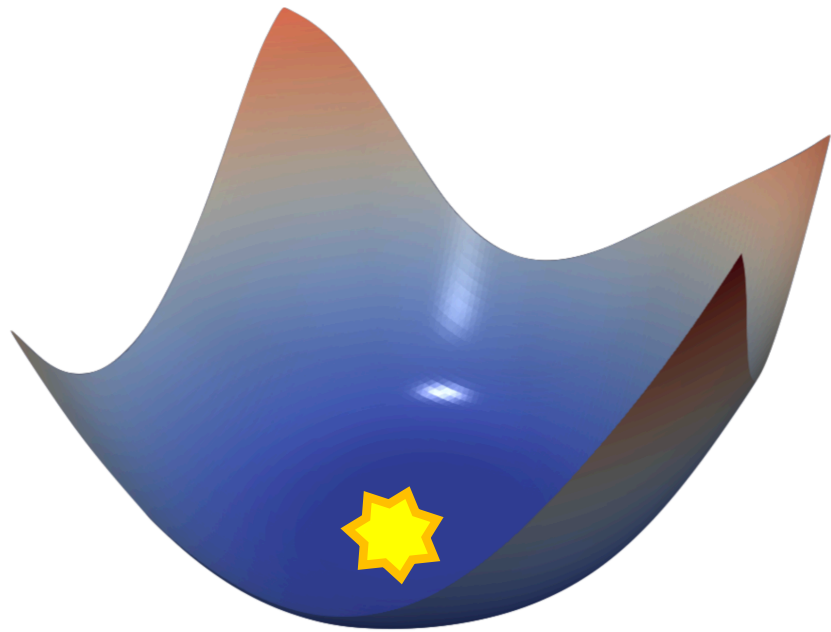




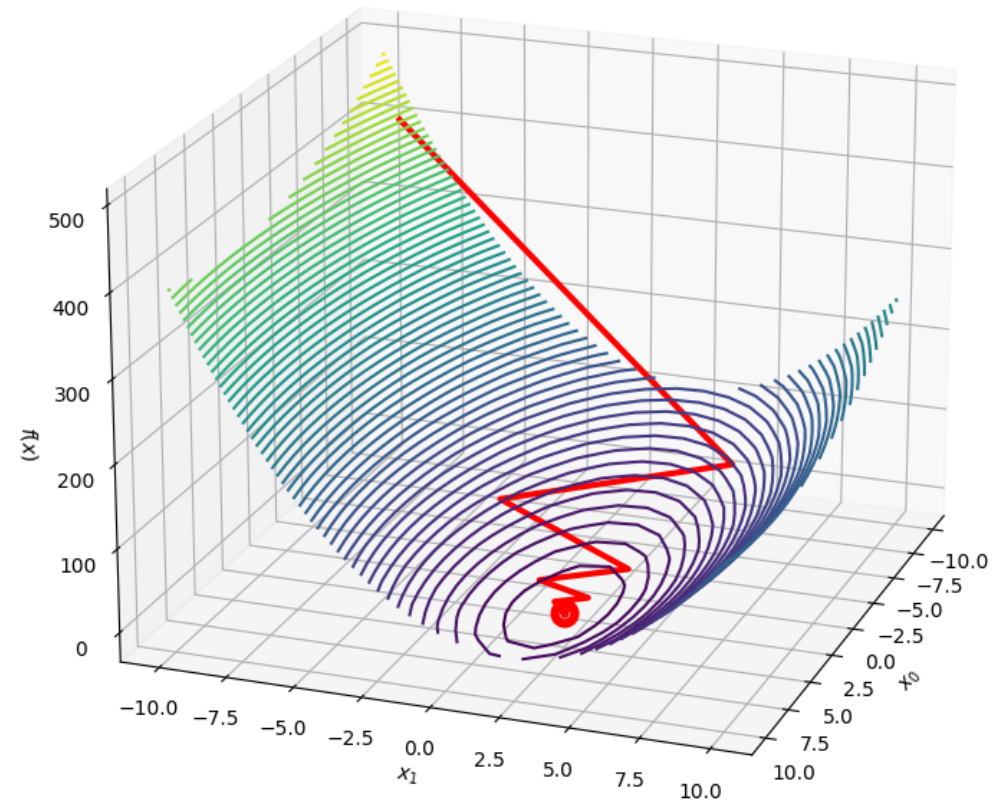
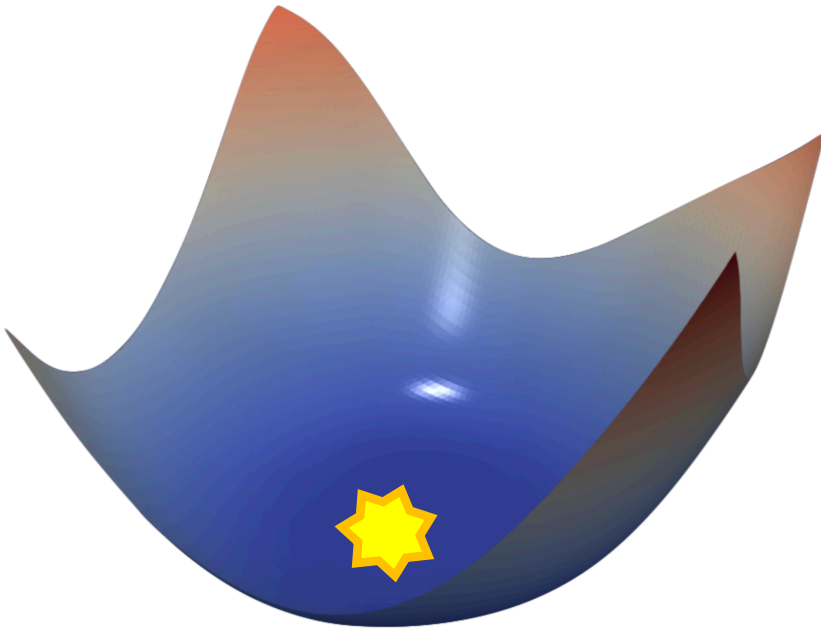
Optimizers

Matteo Bonotto, Ph.D.

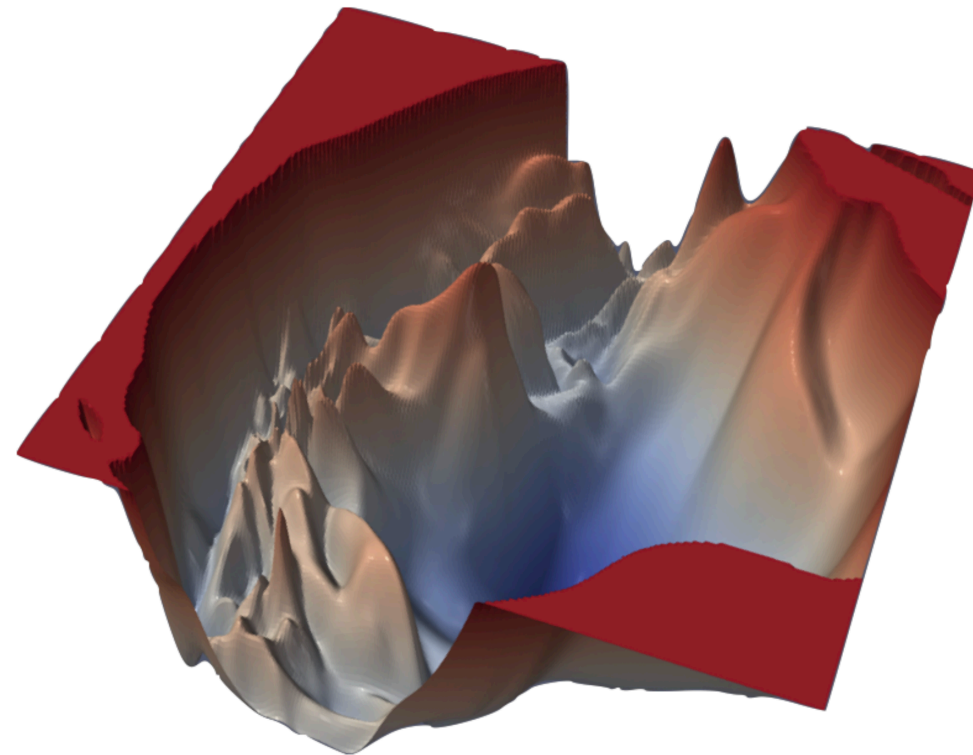
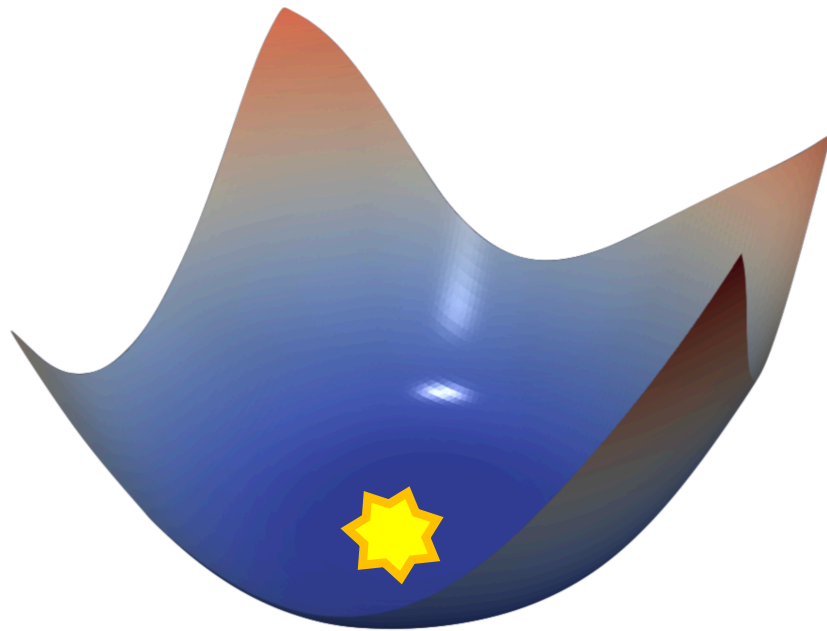
Why do we need an optimizer?



To solve a minimization problem



Sometimes the problem is very hard



Simple 2d problem: the Rosembrock function

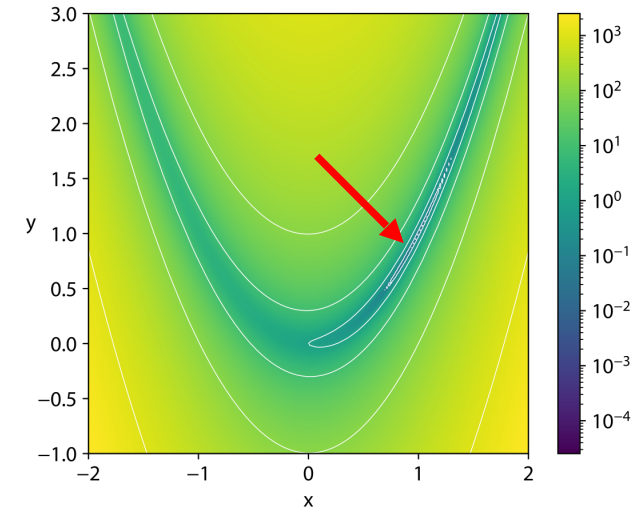
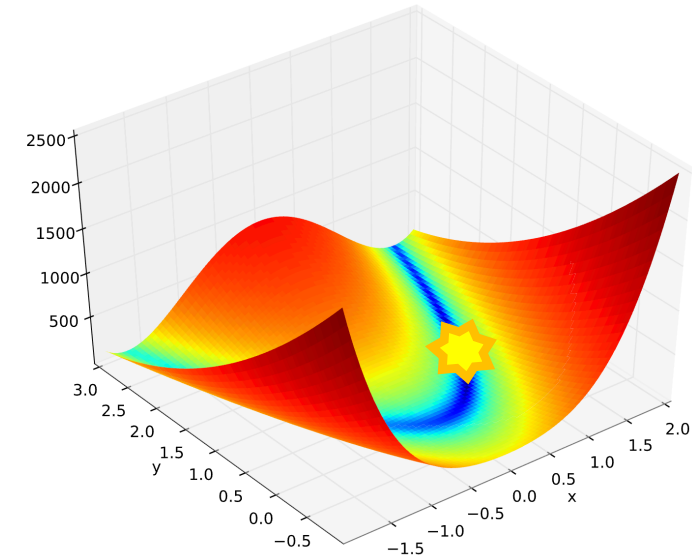
Let's consider a simple example:

- a neural net with only 2 parameters θ
- a loss function $f : \mathcal{R}^2 \rightarrow \mathcal{R}$ mapping θ into a scalar function

We can model this problem by looking at the minimum of the Rosenbrock function

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

where $\theta = (x, y)$



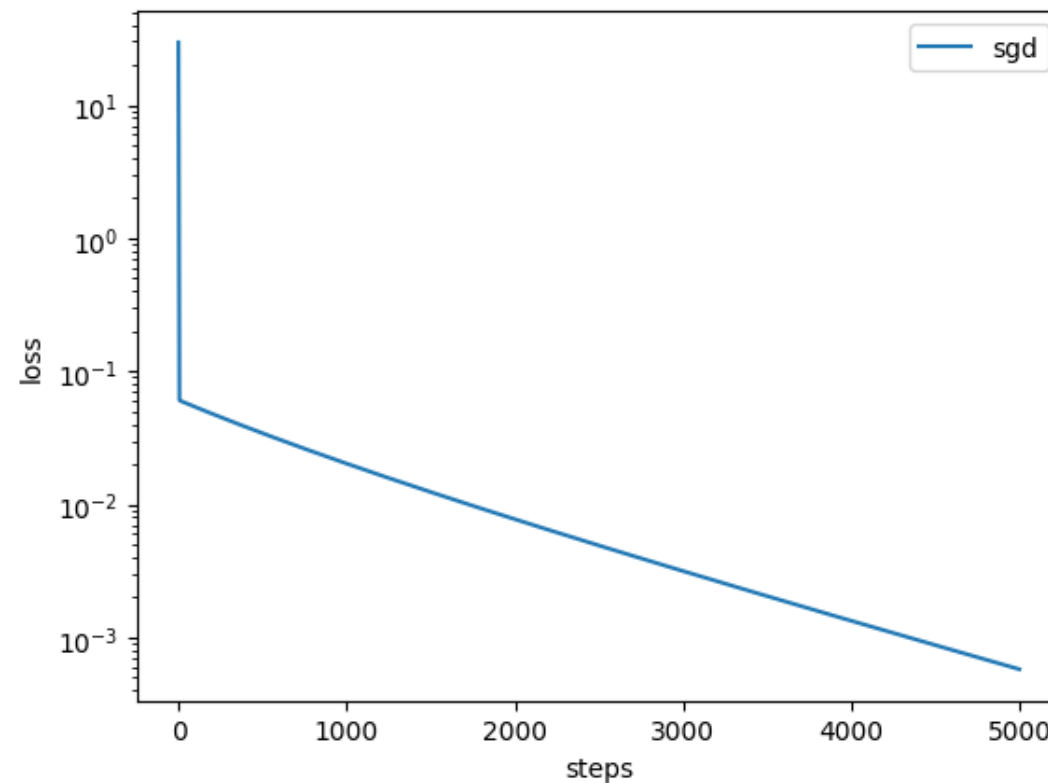
Stochastic Gradient Descent

```
for t = 1, ... do
     $\theta_t \leftarrow \theta_{t-1} - \eta \nabla f_{\theta_{t-1}}$ 
end for
```

```
1 # Gradient Descend (2D)
2 lr = 1e-3
3 theta = np.random.random(size=(2, ))
4 step = 0
5
6 while step < steps:
7     grad_theta = fun_grad(theta) # gradients, shape (N, )
8     theta -= lr * grad_theta
9     step += 1
```

Stochastic Gradient Descent

```
1 # Gradient Descend (2D)
2 lr = 1e-3
3 theta = np.random.random(size=(2, ))
4 step = 0
5
6 while step < steps:
7     grad_theta = fun_grad(theta) # gradients, shape (N, )
8     theta -= lr * grad_theta
9     step += 1
```



SGD with momentum

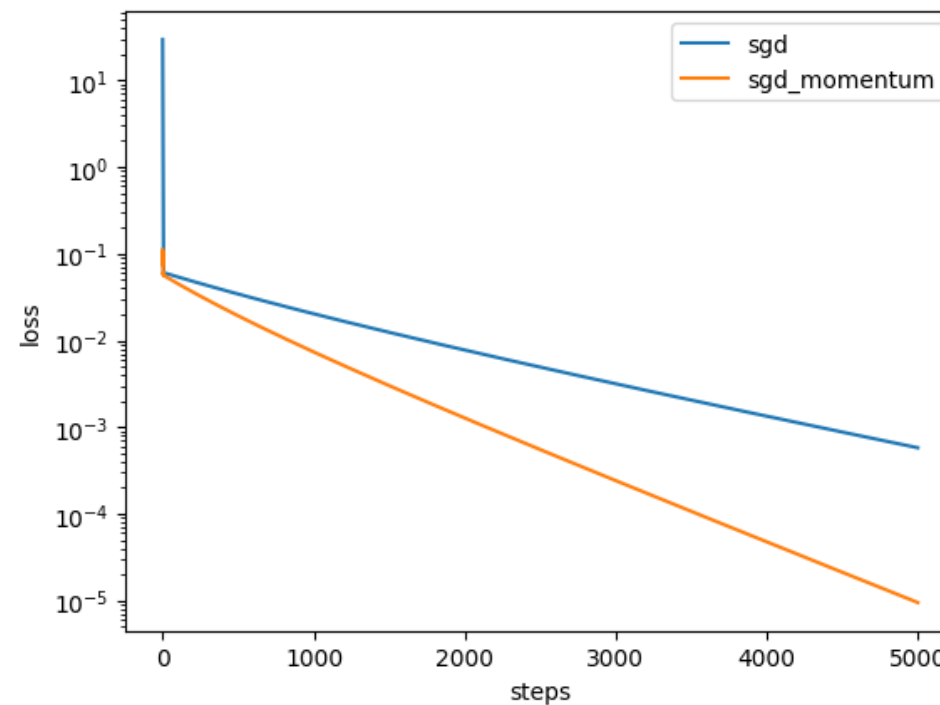
```

$$\Delta\theta_0 \leftarrow 0$$
for  $t = 1, \dots$  do  
     $\Delta\theta_t \leftarrow \alpha\Delta\theta_{t-1} - \eta\nabla f_{\theta_{t-1}}$   
     $\theta_t \leftarrow \theta_{t-1} + \Delta\theta_t$   
end for
```

```
1 # adding momentum: remember the update dtheta at the last step,  
2 # and find the new state as a linear combination between the  
3 # last state and the last update  
4 lr = 1e-3  
5 theta, dtheta = np.random.random(size=(2, )), np.zeros_like(theta)  
6 alpha = 0.5  
7 step = 0  
8  
9 while step < steps:  
10     grad_theta = fun_grad(theta) # gradients, shape (N, )  
11     dtheta = alpha * dtheta - lr * grad_theta # <- optimizer params, shape (N, )  
12     theta += dtheta  
13     step += 1
```


SGD with momentum

```
1 # adding momentum: remember the update dtheta at the last step,  
2 # and find the new state as a linear combination between the  
3 # last state and the last update  
4 lr = 1e-3  
5 theta, dtheta = np.random.random(size=(2, )), np.zeros_like(theta)  
6 alpha = 0.5  
7 step = 0  
8  
9 while step < steps:  
10     grad_theta = fun_grad(theta) # gradients, shape (N, )  
11     dtheta = alpha * dtheta - lr * grad_theta # <- optimizer params, shape (N, )  
12     theta += dtheta  
13     step += 1
```



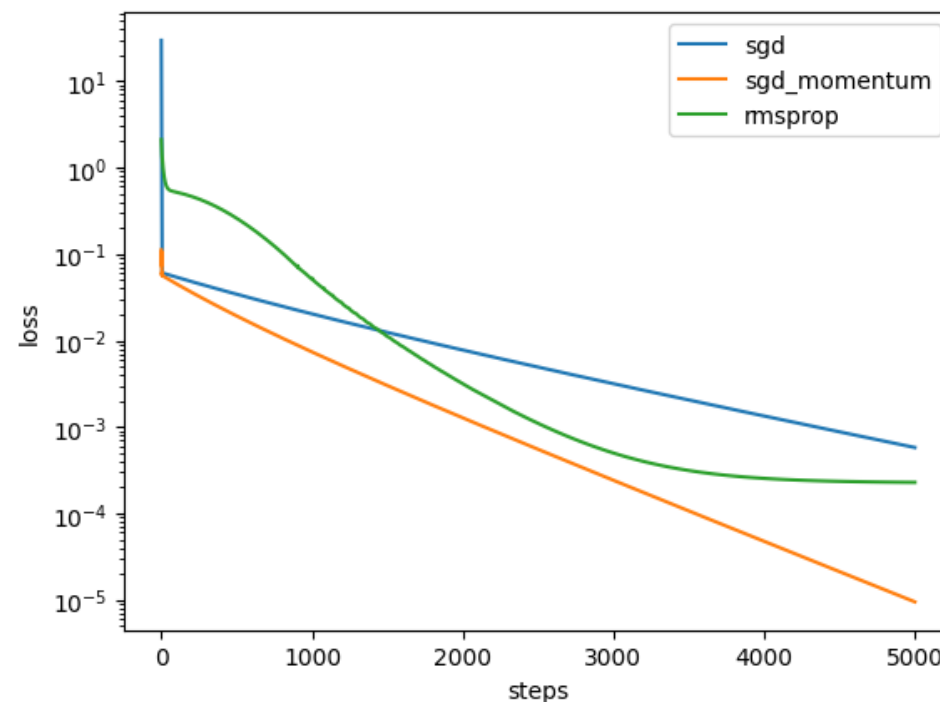
RMSprop: per-parameter learning rate

$$\begin{aligned} v_0 &\leftarrow 0 \\ \text{for } t = 1, \dots \text{ do} \\ &v_t \leftarrow \gamma v_{t-1} + (1 - \gamma)(\nabla f_{\theta_{t-1}})^2 \\ &\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla f_{\theta_{t-1}} \\ \text{end for} \end{aligned}$$

```
1 # RMSProp: divide the learning rate by a factor that is the running
2 # average of the magnitudes of recent gradients for that weight
3 lr = 1e-3
4 theta, v = np.random.random(size=(2, )), np.zeros_like(theta)
5 gamma: float = 0.99
6 step = 0
7
8 while step < steps:
9     grad_theta = fun_grad(theta) # gradients, shape (N, )
10    v = gamma * v + (1 - gamma) * grad_theta ** 2 # <- optimizer params, shape (N, )
11
12    theta -= lr * grad_theta / (np.sqrt(v) + eps)
13    step += 1
```

RMSprop: per-parameter learning rate

```
1 # RMSProp: divide the learning rate by a factor that is the running
2 # average of the magnitudes of recent gradients for that weight
3 lr = 1e-3
4 theta, v = np.random.random(size=(2, )), np.zeros_like(theta)
5 gamma: float = 0.99
6 step = 0
7
8 while step < steps:
9     grad_theta = fun_grad(theta) # gradients, shape (N, )
10    v = gamma * v + (1 - gamma) * grad_theta ** 2 # <- optimizer params, shape (N, )
11
12    theta -= lr * grad_theta / (np.sqrt(v) + eps)
13    step += 1
```



ADAM: momentum + running average

```

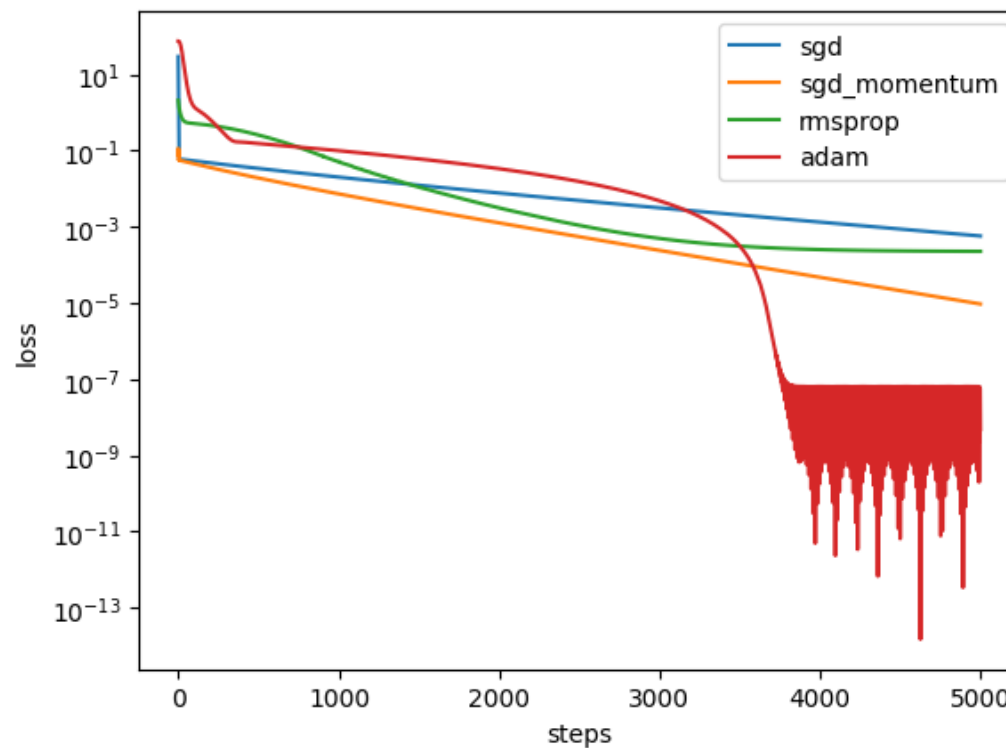
$$\begin{aligned} v_0 &\leftarrow 0 \\ m_0 &\leftarrow 0 \\ \text{for } t = 1, \dots \text{ do} \\ & m_t \leftarrow \beta_m m_{t-1} + (1 - \beta_m)(\nabla f_{\theta_{t-1}}) \\ & v_t \leftarrow \beta_v v_{t-1} + (1 - \beta_v)(\nabla f_{\theta_{t-1}})^2 \\ & \hat{m}_t = \frac{m_t}{1 - \beta_m^t} \\ & \hat{v}_t = \frac{v_t}{1 - \beta_v^t} \\ & \theta_t \leftarrow \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \\ \text{end for} \end{aligned}$$

```

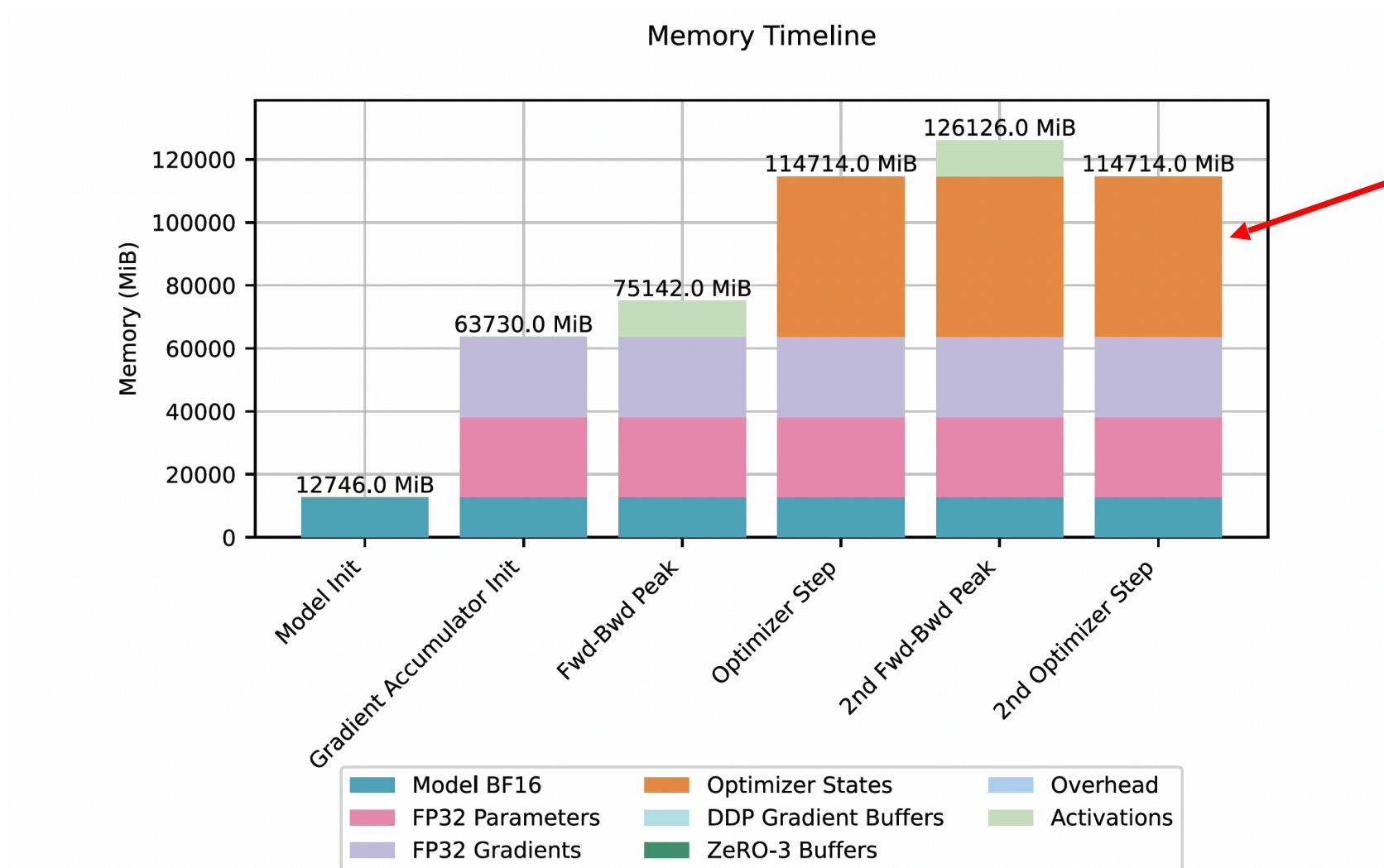
```
1 # Adam: combine RMSprop with momentum.
2 lr = 2e-4
3 theta = np.random.random(size=(2, ))
4 v, m = np.zeros_like(theta), np.zeros_like(theta)
5 beta_v, beta_m = 0.9, 0.9
6 step = 0
7
8 while step < steps:
9     grad_theta = fun_grad(theta) # gradients, shape (N, )
10    m = beta_m * m + (1 - beta_m) * grad_theta # <- optimizer params, shape (N, )
11    v = beta_v * v + (1 - beta_v) * grad_theta ** 2 # <- optimizer params, shape (N, )
12
13    v /= (1 - beta_v ** (step+1))
14    m /= (1 - beta_m ** (step+1))
15
16    theta -= lr * m / (np.sqrt(v) + eps)
17    step += 1
```


ADAM: momentum + running average

```
1 # Adam: combine RMSprop with momentum.
2 lr = 2e-4
3 theta = np.random.random(size=(2, ))
4 v, m = np.zeros_like(theta), np.zeros_like(theta)
5 beta_v, beta_m = 0.9, 0.9
6 step = 0
7 |
8 while step < steps:
9     grad_theta = fun_grad(theta) # gradients, shape (N, )
10    m = beta_m * m + (1 - beta_m) * grad_theta # <- optimizer params, shape (N, )
11    v = beta_v * v + (1 - beta_v) * grad_theta ** 2 # <- optimizer params, shape (N, )
12
13    v /= (1 - beta_v ** (step+1))
14    m /= (1 - beta_m ** (step+1))
15
16    theta -= lr * m / (np.sqrt(v) + eps)
17    step += 1
```



CUDA OOM: optimizer matters



(Bonus) Newton's method: full 2nd order

```
for t = 1, ... do
     $\theta_t \leftarrow \theta_{t-1} - [H(f_{\theta_{t-1}})]^{-1} J(f_{\theta_{t-1}})$ 
end for
```

```
1 def rosebrock_2d_hessian(theta:ArrayLike, a:float=1, b:float=100.) -> ArrayLike:
2     x, y = theta
3     hess_11 = 2 - 4 * b * (y - 3 * x ** 2)
4     hess_12 = - 4 * b * x
5     hess_21 = hess_12
6     hess_22 = 2 * b
7     return np.array([[hess_11, hess_12], [hess_21, hess_22]])
```

```
1 # Newton: full 2nd order method
2 lr = 2e-4
3 theta = np.random.random(size=(2, ))
4 step = 0
5 steps = 10
6
7 while step < steps:
8     grad_theta = fun_grad(theta) # gradients, shape (N, )
9     hess_theta = rosebrock_2d_hessian(theta) # <- hessian, shape (N, N)
10
11     theta -= np.linalg.solve(hess_theta, grad_theta)
12     step += 1
```

(Bonus) Newton's method: full 2nd order

```
1 # Newton: full 2nd order method
2 lr = 2e-4
3 theta = np.random.random(size=(2, ))
4 step = 0
5 steps = 10
6
7 while step < steps:
8     grad_theta = fun_grad(theta) # gradients, shape (N, )
9     hess_theta = rosenbrock_2d_hessian(theta) # <- hessian, shape (N, N)
10
11     theta -= np.linalg.solve(hess_theta, grad_theta)
12     step += 1
```

```
step=0, theta=array([0.832, 0.212]), 23.1
step=1, theta=array([0.834, 0.696]), 0.0275
step=2, theta=array([1. , 0.972]), 0.0754
step=3, theta=array([1. , 1.]), 6.97e-09
step=4, theta=array([1. , 1.]), 4.85e-15
step=5, theta=array([1. , 1.]), 0.0
```

